

HDOC Day-30

Array Operations Using a Menu-Driven C Program

Aim

To write a menu-driven C program to perform different operations on a given array using switch-case statements, loops, and functions. The program performs sorting with Binary Search, insertion and deletion, right rotation, and exit operations.

Algorithm

1. Start the program.
2. Input the size and elements of the array.
3. Display the menu: Bubble Sort and Binary Search; Insertion and Deletion; Right Rotation by k Positions; Exit.
4. Accept the user choice and use a switch statement to perform the selected operation.
5. For Bubble Sort and Binary Search, sort the array in ascending order, display it, accept a search key, and perform Binary Search.
6. For Insertion and Deletion, display a submenu. Insert by shifting elements right, or delete by shifting remaining elements left. Validate the position.
7. For Right Rotation, accept k, rotate the array to the right by k positions, and display the result.
8. Display an error message for an invalid menu choice or invalid position.
9. Continue displaying the menu until Exit is selected.
10. Stop the program.

C Program

```
#include <stdio.h>

#define MAX 100

void displayArray(int arr[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
```

```

    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1, mid;
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main()
{
    int arr[MAX];
    int n, i, choice, subChoice;
    int key, position, element;
    int k, temp;

    printf("Enter size of array: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    do
    {
        printf("\n----- ARRAY OPERATIONS MENU -----\\n");
        printf("1. Bubble Sort and Binary Search\\n");
        printf("2. Insertion and Deletion\\n");
        printf("3. Right Rotation by k Positions\\n");
        printf("4. Exit\\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                bubbleSort(arr, n);
                printf("Sorted Array: ");
                displayArray(arr, n);
                printf("Enter element to search: ");
                scanf("%d", &key);
                position = binarySearch(arr, n, key);
                if (position != -1)
                    printf("Element found at position %d.\\n", position + 1);
                else
                    printf("Element not found.\\n");
                break;

            case 2:
                printf("\\n----- INSERTION / DELETION -----\\n");
                printf("1. Insert an Element\\n");
                printf("2. Delete an Element\\n");

```

```

printf("Enter your choice: ");
scanf("%d", &subChoice);

switch (subChoice)
{
    case 1:
        if (n >= MAX) { printf("Array is full.\n"); break; }
        printf("Enter element to insert: ");
        scanf("%d", &element);
        printf("Enter position (1 to %d): ", n + 1);
        scanf("%d", &position);
        if (position < 1 || position > n + 1)
        { printf("Invalid position!\n"); break; }
        for (i = n; i >= position; i--)
            arr[i] = arr[i - 1];
        arr[position - 1] = element;
        n++;
        printf("Array after insertion: ");
        displayArray(arr, n);
        break;

    case 2:
        if (n == 0) { printf("Array is empty.\n"); break; }
        printf("Enter position to delete (1 to %d): ", n);
        scanf("%d", &position);
        if (position < 1 || position > n)
        { printf("Invalid position!\n"); break; }
        for (i = position - 1; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
        printf("Array after deletion: ");
        displayArray(arr, n);
        break;

    default:
        printf("Invalid submenu choice!\n");
}
break;

case 3:
    printf("Enter number of positions to rotate: ");
    scanf("%d", &k);
    if (n == 0) { printf("Array is empty.\n"); break; }
    k = k % n;
    while (k > 0)
    {
        temp = arr[n - 1];
        for (i = n - 1; i > 0; i--)
            arr[i] = arr[i - 1];
        arr[0] = temp;
        k--;
    }
    printf("Array after right rotation: ");
    displayArray(arr, n);
    break;

case 4:
    printf("Exiting program...\n");
    break;

default:
    printf("Invalid choice! Please enter a number from 1 to 4.\n");
}
} while (choice != 4);
return 0;
}

```

Test Case Table

Test Case	Operation	Input	Expected Result
1	Bubble Sort + Search	Array: 5 2 8 1 4; Search: 8	Sorted: 1 2 4 5 8; found at position 5
2	Binary Search	Array: 7 3 9 1 5; Search: 6	Sorted: 1 3 5 7 9; element not found
3	Insertion	10 20 30 40; insert 25 at position 3	10 20 25 30 40
4	Deletion	10 20 30 40; delete position 2	10 30 40
5	Invalid Position	Insert at position 8 in a 4-element array	Invalid position!
6	Right Rotation	1 2 3 4 5; k = 2	4 5 1 2 3
7	Right Rotation	10 20 30 40; k = 1	40 10 20 30
8	Invalid Menu	Choice 5	Invalid choice message
9	Exit	Choice 4	Program exits

Result

The menu-driven C program was successfully implemented to perform the required array operations. The program sorts array elements using Bubble Sort and performs Binary Search, supports insertion and deletion at valid positions, and rotates the array to the right by a specified number of positions. It also handles invalid positions and menu choices appropriately.